

Agile Estimator v2

Documentacao Tecnica Detalhada

Versao: 2.0

Responsavel tecnico: Savio Mendes

Data do documento: 15/06/2026

Repositorio: [rodolfobdossantos/agile_estimator_ofc](https://github.com/rodolfobdossantos/agile_estimator_ofc) (GitHub)

API em producao: <https://agile-estimator-ofc.onrender.com>

MLflow UI (experimentos): <https://agile-estimator-mlflow.onrender.com>

Marcos cobertos por este documento

Marco 1:	Engenharia de Dados	14/03/2026
Marco 2:	Modelagem de Machine Learning	26/03/2026
Marco 3:	Servico de Previsao (API)	06/04/2026
Marco 4:	Infraestrutura e Deploy	16/04/2026
Marco 5:	Integracao Streamlit + Documentacao Tecnica	Fase 2

Arquitetura Geral



Pagina 2 | Gerado em 15/06/2026

Marco 1 - Engenharia de Dados

Data de entrega: 14/03/2026

Status: ENTREGUE

1.1 Fonte de Dados

O sistema foi treinado com o dataset Maxwell, coletado por K.D. Maxwell ("Applied Statistics for Software Managers", 2002). O dataset contem projetos reais de desenvolvimento de software em empresas de telecomunicacoes. O repositorio contem dados brutos de quatro datasets publicos de benchmark de esforco em software (pasta model/data/raw/):

Dataset	Arquivo	Projetos	Uso no v2
Maxwell	maxx.csv	62	Dataset de treino (unico)
China	china.csv	499	Explorado na EDA; nao usado no modelo final
Desharnais	desh.csv	81	Explorado na EDA; nao usado no modelo final
NASA93	nasa93.csv	93	Explorado na EDA; nao usado no modelo final
ISBSG	EBSPM_Research_Repository_v07072017.csv	varios	Explorado; nao usado no modelo final

A escolha do Maxwell como unico dataset de treino foi motivada pela consistencia das variaveis disponiveis: tamanho funcional (Function Points / AFP) e fatores de ajuste tecnico (TCA) em escala Likert, diretamente alinhados com a abordagem IFPUG de estimativa.

1.1.1 Analise Comparativa e Criterios de Selecao

Durante a fase de Engenharia de Dados (Marco 1), todos os cinco datasets foram carregados, inspecionados e avaliados quanto a compatibilidade com a abordagem IFPUG adotada no projeto. A tabela abaixo resume os criterios de avaliacao e justifica a escolha do Maxwell como dataset exclusivo de treino:

Dataset	Metrica de Tamanho	Fatores de Complexidade	Compatibilidade IFPUG	Decisao
Maxwell	AFP (Adjusted FP)	4 TCAs em escala Likert 1-5	Total	SELECIONADO
China	AFP	Ausentes / esquema diferente	Parcial	Descartado
Desharnais	UFP (nao ajustado)	Schema de complexidade incompativel	Baixa	Descartado
NASA93	KDSI (linhas)	COCOMO (VL/L/N/H/VH) - nao Likert	Nenhuma	Descartado
ISBSG	Misto (FP/SNAP)	Inconsistente entre projetos	Baixa	Descartado

O Maxwell foi o unico dataset que ofereceu simultaneamente: (1) tamanho funcional em AFP compativel com IFPUG, (2) fatores de ajuste tecnico (T03, T09, T11, T15) em escala Likert 1-5 sem valores ausentes, e (3) variavel-alvo effort_hours em horas-pessoa brutas. Os demais datasets foram explorados em notebooks de EDA disponíveis em model/notebooks/inferencia/agile_v2/data_modelling.ipynb, que registra a analise comparativa completa com os criterios de descarte de cada dataset.

1.2 Colunas do Dataset Maxwell e Mapeamento

Coluna Original	Nome Canonico	Tipo	Descricao
Size	function_points	Continua	Adjusted Function Points (AFP) - tamanho funcional
Effort	effort_hours	Continua	Esforco total em horas-pessoa (variavel-alvo)
T03	performance_requirements	Likert 1-5	Exigencia de desempenho e velocidade
T09	complex_processing	Likert 1-5	Complexidade tecnica do processamento

T11	installation_ease	Likert 1-5	Facilidade de instalacao e deploy (5=muito facil)
T15	additional_complexity_factor	Likert 1-5	Fatores adicionais de complexidade

1.3 Pipeline de Limpeza e Pre-processamento

1.3.1 Remocao de Outliers (IQR)

Outliers na variavel-alvo effort_hours foram removidos pelo metodo Tukey (IQR):

<pre>Q3 = df['effort_hours'].quantile(0.75) IQR = df['effort_hours'].quantile(0.75) - df['effort_hours'].quantile(0.25) df = df[df['effort_hours'] <= Q3 + 1.5 * IQR] # Resultado: 62 projetos originais -> 57 projetos apos limpeza</pre>	
Metrica	Valor
Projetos originais	62
Projetos apos limpeza	57
Outliers removidos	5
effort_hours minimo	583 horas
effort_hours maximo (apos)	18.500 horas
Mediana	4.557 horas
Media	5.653 horas

1.3.2 Transformacao da Variavel-Alvo

A variavel effort_hours foi transformada com logaritmo natural para normalizar a distribuicao assimetrica e estabilizar a variancia:

<pre>y = np.log(df['effort_hours']) # log-transform # Na inferencia, a API desfaz a transformacao: effort_hours_previsto = np.exp(model.predict(X)[0])</pre>
--

1.3.3 Padronizacao dos 4 Fatores Likert

Os 4 fatores de complexidade tecnica (T03, T09, T11, T15) foram padronizados com StandardScaler antes da aplicacao do PCA. O scaler foi fitado nos 57 projetos Maxwell e salvo como artefato de producao:

<pre>from sklearn.preprocessing import StandardScaler scaler = StandardScaler() X_scaled = scaler.fit_transform(df[['performance_requirements', 'complex_processing', 'installation_ease', 'additional_complexity_factor']]) joblib.dump(scaler, 'api/artifacts/preprocessing/scaler_pca_features.pkl')</pre>		
Feature	Mean (media)	Scale (desvio padrao)
performance_requirements	3.0000	0.8983
complex_processing	4.0351	0.7484
installation_ease	3.3158	0.9395
additional_complexity_factor	3.2982	0.7484

1.3.4 Reducao de Dimensionalidade - PCA

PCA com 2 componentes foi aplicado sobre os 4 fatores Likert padronizados. Os 2 componentes explicam ~70,6% da variancia total:

Componente	Variância Explicada	Interpretação	Principais Cargas
PC1	44,37%	Complexidade técnica geral	complex_processing(0.61), add_complexity(0.55), install_ease(0.50)
PC2	26,25%	Tradeoff performance vs. instalação	perf_requirements(0.81), install_ease(-0.51), complex_processing(-0.15)

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
pca.fit(X_scaled)
# explained_variance_ratio_: [0.4437, 0.2625] => 70.62% total
joblib.dump(pca, 'api/artifacts/preprocessing/pca_2.pkl')
```

1.3.5 Padronização de function_points

O campo function_points (Size/AFP) foi padronizado separadamente. O dataset processado já contém function_points padronizado, portanto a inferência deve aplicar a mesma transformação antes de enviar a API:

```
# Parametros extraídos de scaler_maxx.pkl (índice 0 = function_points)
fp_mean = 514.8596 # media nos 57 projetos Maxwell
fp_scale = 516.2373 # desvio padrao

fp_std = (function_points_raw - fp_mean) / fp_scale
```

1.4 Dataset Processado

O dataset final (model/data/processed/maxx_processed.csv) contém 57 linhas e 14 colunas:

- function_points - padronizado
- effort_hours - log(esforço original)
- customer_participation, logical_complexity, requirements_volatility, requirements_quality, productivity_factors, installation_ease, team_technical_capability, individual_skill - outros fatores Maxwell padronizados
- PC1, PC2, PC3, PC4 - componentes PCA (somente PC1 e PC2 entram no modelo)

1.5 Artefatos Gerados (Marco 1)

Artefato	Caminho	Descrição
scaler_pca_features.pkl	api/artifacts/preprocessing/	StandardScaler fitado nos 4 fatores Likert
pca_2.pkl	api/artifacts/preprocessing/	PCA n=2 fitado nos 4 fatores padronizados
scaler_maxx.pkl	api/artifacts/preprocessing/	Scaler Maxwell; índice 0 = function_points
maxx_processed.csv	model/data/processed/	Dataset limpo e transformado, 57 x 14

Marco 2 - Modelagem de Machine Learning

Data de entrega: 26/03/2026

Status: ENTREGUE

2.1 Selecao de Features

Apos analise de correlacao de Pearson ($|r| > 0.20$ com effort_hours) e reducao por PCA, foram selecionadas 3 features para o modelo final:

Feature	Origem	Descricao
function_points	Maxwell - coluna Size	Tamanho funcional (AFP) - padronizado
PC1	PCA sobre T03,T09,T11,T15	Complexidade tecnica geral (~44% variancia)
PC2	PCA sobre T03,T09,T11,T15	Tradeoff performance vs. instalacao (~26% variancia)

As demais colunas do dataset processado (customer_participation, logical_complexity, etc.) são artefatos de um pipeline exploratório anterior e não entram no modelo.

2.2 Algoritmos Avaliados

Algoritmo	Regularizar	Grid de Hiperparametros	Observacao
Ridge	L2	alpha: [0.01, 0.1, 1, 10]	Baseline linear regularizado
Lasso	L1	alpha: [0.001, 0.01, 0.1, 1]	Selecao implicita de features
SVR	-	C: [0.1,1,10], gamma: [scale,auto]	Pipeline com StandardScaler adicional
RandomForest	-	n_estimators: [100,200,300], max_depth: varios	Ensemble; risco de overfitting com n=57
GradientBoosting	-	n_estimators: [100,200,300,400], lr: [0.05..0.3]	Boosting sequencial

2.3 Estrategia de Validacao Cruzada

Com apenas 57 observacoes, uma unica divisao treino/teste produziria estimativas instáveis. Foi utilizado RepeatedKFold para maximizar a estabilidade da avaliacao:

```
from sklearn.model_selection import RepeatedKFold, GridSearchCV

cv = RepeatedKFold(n_splits=5, n_repeats=10, random_state=42)
# => 50 avaliações independentes por modelo
# => cada fold: 80% treino (45 projetos) / 20% teste (12 projetos)

grid = GridSearchCV(
    pipeline, param_grid, cv=cv,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)
```

2.4 Resultados Comparativos (holdout 20%)

Modelo	MAE (h)	RMSE (h)	R2	MAPE (%)	Posicao
Lasso	1.616	1.877	0.378	48.9%	1 - SELECIONADO
RandomForest	1.914	2.213	0.136	44.2%	2
Ridge	1.879	2.214	0.136	52.7%	3
GradientBoosting	1.962	2.261	0.099	43.3%	4
SVR	2.810	3.589	-1.27	70.0%	5

O Lasso foi selecionado por apresentar o menor RMSE absoluto no holdout, o melhor R2 (0.378) e o menor MAE. O MAPE de 48.9% e característico de estimativas de esforço em software - a variabilidade intrínseca dos projetos impõe um limite superior de precisão mesmo com modelos sofisticados.

2.5 Modelo Selecionado - Lasso

Atributo	Valor
Algoritmo	sklearn.linear_model.Lasso
Alpha (melhor)	0.1 (selecionado por GridSearchCV)
Coeficiente - function_points	0.3409
Coeficiente - PC1	0.1666
Coeficiente - PC2	-0.2723
Intercept	8.3585 => $\exp(8.3585) = 4.254$ horas (média Maxwell)
Target de treino	$\log(\text{effort_hours})$
Predição final	$\text{np.exp}(\text{model.predict(X)}[0]) \Rightarrow \text{horas-pessoa}$

Interpretação dos coeficientes:

- function_points (+0.341): aumento de 1 desvio-padrão no AFP => $\exp(0.341) = 1.41x$ mais esforço
- PC1 (+0.167): maior complexidade técnica geral => mais esforço
- PC2 (-0.272): PC2 captura contraste performance vs. facilidade de instalação

2.6 Rastreamento com MLflow

Todos os experimentos foram rastreados com MLflow em `model/experiments/mlruns/`. Para cada algoritmo foram registrados: hiperparâmetros do GridSearchCV, métricas no holdout (MAE, RMSE, R2, MAPE_percent) e o artefato .pkl do `best_estimator_`.

```
import mlflow, mlflow.sklearn

mlflow.set_tracking_uri('file:./model/experiments/mlruns')
mlflow.set_experiment('agile_estimator_modeling')

with mlflow.start_run(run_name=name):
    grid.fit(X_train, y_train)
    best_model = grid.best_estimator_ # modelo com alpha=0.1 correto
    mlflow.log_params(grid.best_params_)
    mlflow.log_metric('RMSE', rmse)
    mlflow.sklearn.log_model(best_model, name)
```

ATENÇÃO: o notebook foi corrigido para usar `grid.best_estimator_` ao salvar o artefato final. O bug original re-fazia o fit com `alpha=1.0` (padrão sklearn), zerando todos os coeficientes Lasso.

MLflow UI em Produção (entrega adicional)

Como entrega adicional, o histórico completo de experimentos MLflow está publicado e acessível via browser, sem necessidade de instalar nada localmente. O serviço roda em um container Docker separado, hospedado no Render.com:

Atributo	Detalhe
URL pública	https://agile-estimator-mlflow.onrender.com
O que pode ser visto	Todos os runs de treino (Ridge, Lasso, SVR, RF, GBM), hiperparâmetros do GridSearchCV, métricas MAE/RMSE/R2/MAPE, artefatos .pkl de cada experimento e comparativo visual entre algoritmos
Infraestrutura	Docker (mlflow_server/Dockerfile) + Render Web Service

Autenticacao	Nenhuma -- acesso publico somente leitura
Cold start	~30s apos inatividade (plano gratuito)

O MLflow UI permite ao cliente visualizar de forma transparente todo o processo de selecao do modelo: quais algoritmos foram avaliados, com quais parametros, quais metricas cada um obteve e por que o Lasso foi selecionado. Esta transparencia e um diferencial importante para auditorias e para futuras decisoes de retreinamento.

2.7 Interpretabilidade com SHAP

Foi utilizado SHAP (SHapley Additive exPlanations) para interpretar as contribuicoes de cada feature no conjunto de teste. O grafico de barras mostra a importância media absoluta de cada feature; o summary plot mostra a direcao do efeito (valores altos vs. baixos de cada feature).

```
import shap
explainer = shap.Explainer(best_model.predict, X_train)
shap_values = explainer(X_test)
shap.plots.bar(shap_values) # importancia media
shap.summary_plot(shap_values, X_test) # direcao do efeito
```

2.8 Artefatos Gerados (Marco 2)

Artefato	Caminho	Descricao
agile_estimator_v2.pkl	api/artifacts/model/	Pipeline Lasso - alpha=0.1, coef=[0.341, 0.167, -0.272]
agile_estimator_v2.pkl	model/artifacts/model/	Copia para o repositorio de modelagem
features.json	api/artifacts/metadata/	Metadados do modelo: features, pipeline, metricas
mlruns/	model/experiments/mlruns/	Experimentos MLflow com todos os runs e modelos
modelagem-ml.ipynb	model/notebooks/treino/agile_v2/	Notebook de treino documentado

Marco 3 - Servico de Previsao (API)

Data de entrega: 06/04/2026

Status: ENTREGUE

3.1 Estrutura do Projeto API

```
api/
+-- app/
|   +-- main.py          # FastAPI app, CORS, rotas /predict e /health
|   +-- predict.py       # Logica de inferencia e selecao de features
|   +-- model_loader.py  # Carrega o .pkl mais recente de artifacts/model/
|   +-- feature_loader.py # Le features.json para selecao dinamica de features
|   +-- schemas.py       # Pydantic: PredictionInput (function_points, PC1, PC2)
|   +-- logger.py        # Logger estruturado (formato ISO 8601)
+-- artifacts/
    +-- model/
    |   +-- agile_estimator_v2.pkl
    +-- preprocessing/
    |   +-- pca_2.pkl
    |   +-- scaler_pca_features.pkl
    |   +-- scaler_maxx.pkl
    +-- metadata/
        +-- features.json
```

3.2 Endpoints

Endpoint	Metodo	Descricao	Autenticacao
POST /predict	POST	Inferencia - retorna esforco em horas-pessoa	Nenhuma
GET /health	GET	Health check com 3 verificacoes automaticas	Nenhuma
GET /docs	GET	Swagger UI gerado automaticamente pelo FastAPI	Nenhuma

3.3 Schema de Entrada e Saida

```
# Entrada (Pydantic - schemas.py)
class PredictionInput(BaseModel):
    function_points: float # ja padronizado: (fp_raw - 514.86) / 516.24
    PC1: float             # 1a componente PCA dos 4 fatores Likert
    PC2: float             # 2a componente PCA dos 4 fatores Likert

# Saida
# {"prediction": 4982.3} # horas-pessoa (ja transformado com np.exp())

# Exemplo de chamada:
curl -X POST https://agile-estimator-ofc.onrender.com/predict \
  -H "Content-Type: application/json" \
  -d '{"function_points": -0.029, "PC1": 0.52, "PC2": -0.31}'
```

3.4 Fluxo de Inferencia (predict.py)

```
def make_prediction(model, data: PredictionInput) -> float:
    features = ['function_points', 'PC1', 'PC2']
    df = pd.DataFrame([data.dict()])[features]
```

Pagina 9 | Gerado em 15/06/2026

```
prediction = model.predict(df) # retorna log(effort_hours)
return float(np.exp(prediction[0])) # desfaz o log => horas-pessoa
```

3.5 Carregamento Dinâmico do Modelo (model_loader.py)

O `model_loader` carrega automaticamente o arquivo `.pkl` mais recente da pasta `artifacts/model/`. Isso permite substituir o modelo sem alterar código:

```
def get_latest_model():
    files = [f for f in os.listdir(MODEL_DIR) if f.endswith('.pkl')]
    latest = max(files, key=lambda x: os.path.getctime(...))
    return os.path.join(MODEL_DIR, latest)

model = load_model() # carregado no startup do FastAPI
```

3.6 Endpoint de Health Check (GET /health)

O health check executa 3 verificações a cada chamada:

Verificação	O que checa	Campo na resposta
1. Modelo em memória	model is not None	model_loaded: bool
2. Arquivo no disco	os.path.exists(model_path)	model_file_exists: bool
3. Inferência de sanidade	predict([[1, 0.0, 0.0]]) sem exceção	inference_ok: bool
4. Tempo de resposta	Tempo total do health check	response_time_ms: float

```
# Resposta esperada quando tudo ok:
{
  "status": "ok",
  "checks": {
    "model_loaded": true,
    "model_file_exists": true,
    "inference_ok": true,
    "test_prediction": 8.358,
    "response_time_ms": 12.4
  }
}
```

3.7 Logging

Logs estruturados via módulo `logger.py` usando o módulo padrão `logging` do Python:

```
# Formato: YYYY-MM-DD HH:MM:SS,mmm - LEVEL - mensagem
# Exemplo de saída:
2026-05-11 14:32:01,123 - INFO - Prediction: 4982.3
2026-05-11 14:32:15,456 - ERROR - Error: Connection timeout
```

3.8 Dependências da API (requirements.txt)

```
fastapi
uvicorn[standard]
pandas
scikit-learn
joblib
mlflow
```

Marco 4 - Infraestrutura e Deploy

Data de entrega: 16/04/2026

Status: ENTREGUE

4.1 Containerizacao com Docker

A API foi containerizada usando Docker com imagem base python:3.11-slim, garantindo reproducibilidade e portabilidade entre ambientes.

Dockerfile

```
FROM python:3.11-slim

# Evita criacao de .pyc e bufferiza logs em tempo real
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

# Copia requirements primeiro (aproveita cache de camadas Docker)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

# Porta configuravel via variavel de ambiente $PORT (necessario no Render)
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

docker-compose.yml (reinicio automatico em falha)

```
version: "3.9"

services:
  api:
    build: .
    container_name: agile_api
    ports:
      - "8000:10000"
    volumes:
      - .:/app
    restart: always    # reinicia automaticamente em caso de falha
```

A diretiva 'restart: always' garante que o container seja reiniciado automaticamente em caso de crash ou reinicializacao do sistema operacional, sem intervencao manual.

4.2 Deploy em Producao - Render.com

Atributo	Detalhe
Plataforma	Render.com (Web Service)
URL	https://agile-estimator-ofc.onrender.com
HTTPS	Certificado SSL automatico (Let's Encrypt)
Deploy trigger	Push para branch main no GitHub (auto-deploy)

Reinicio	Automatico em caso de crash (Docker restart: always)
Cold start	~30s apos 15min de inatividade (plano gratuito)
Mitigacao cold start	GET /health antes de calcular estimativas

4.3 Fluxo de Deploy

1. Desenvolvedor faz push para branch main no GitHub
2. Render detecta o push via webhook
3. Render faz docker build com o Dockerfile da pasta api/
4. Container e iniciado com o novo artefato .pkl
5. Render realiza health check automatico
6. Trafego e redirecionado para o novo container (zero downtime deploy)

4.4 Rodar Localmente com Docker

```
# Construir e iniciar
cd api
docker-compose up --build

# Testar
curl http://localhost:8000/health
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"function_points": -0.029, "PC1": 0.52, "PC2": -0.31}'

# Swagger UI
# Abrir: http://localhost:8000/docs
```

4.5 Rodar Localmente sem Docker

```
cd api
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
# Acesse: http://localhost:8000/docs
```

Marco 5 - Integração Streamlit e Documentação Técnica

Data de entrega: Fase 2

Status: ENTREGUE

5.1 Adequação da Interface para Consumir a API

A interface Streamlit foi reescrita para não executar nenhum modelo localmente. Todo processamento de inferência ocorre exclusivamente na API REST hospedada no Render.com. O Streamlit é responsável apenas pelo pré-processamento client-side das features brutas antes de enviar o payload a API.

Pipeline Completo de Inferência (client-side)

```
# 1. Coleta de inputs (Manual / CSV / Trello)
row = {'function_points': 500, 'performance_requirements': 3,
       'complex_processing': 4, 'installation_ease': 3,
       'additional_complexity_factor': 3}

# 2. Padronizar function_points (scaler_maxx.pkl, índice 0)
# FP_MEAN=514.8596, FP_SCALE=516.2373 (carregados no startup)
fp_std = (row['function_points'] - FP_MEAN) / FP_SCALE

# 3. Padronizar 4 fatores Likert (scaler_pca_features.pkl)
X = pd.DataFrame([row])[BUSINESS_FEATURES].astype(float)
X_scaled = scaler.transform(X)

# 4. Redução de dimensionalidade (pca_2.pkl)
pcs = pca.transform(X_scaled)[0]
PC1, PC2 = float(pcs[0]), float(pcs[1])

# 5. POST /predict
payload = {"function_points": fp_std, "PC1": PC1, "PC2": PC2}
resp = requests.post(API_URL, json=payload, timeout=60)
effort_h = float(resp.json()['prediction']) # horas-pessoa
```

5.2 Estrutura do Projeto Streamlit

```
streamlit_app/
+-- app.py                # Aplicação principal (600+ linhas)
+-- scripts_app/
    +-- get_public_trello_board.py # Importador de boards públicos Trello
```

5.3 Carregamento de Artefatos no Startup

```
@st.cache_resource
def load_artifacts():
    # scaler_pca_features: 4 fatores Likert
    scaler = joblib.load('api/artifacts/preprocessing/scaler_pca_features.pkl')
    pca = joblib.load('api/artifacts/preprocessing/pca_2.pkl')
    # scaler_maxx[0]: mean e scale de function_points
    scaler_fp = joblib.load('api/artifacts/preprocessing/scaler_maxx.pkl')
    fp_mean = float(scaler_fp.mean_[0]) # 514.8596
    fp_scale = float(scaler_fp.scale_[0]) # 516.2373
    return scaler, pca, fp_mean, fp_scale
```

```
scaler, pca, FP_MEAN, FP_SCALE = load_artifacts()
# @st.cache_resource => carregado 1 vez, reutilizado em todos os reruns
```

5.4 Modos de Entrada de Dados

Modo	Aba	Descrição
Tutorial	Aba 1	Explica o modelo, inputs, outputs, escala Likert e limitações
Input Manual	Aba 2	Formulário com 5 campos; validação Likert 1-5; acumula projetos
Upload CSV	Aba 3	Importação em lote; preview 10 linhas; botão de confirmação
Trello	Aba 4	Importa de board público via URL; sem autenticação

5.5 Calculadora IFPUG Embutida

Disponível como expandir na aba Input Manual. Calcula Adjusted Function Points com os pesos padrão IFPUG:

Tipo de Função	Simples	Média	Complexa
EI - External Input	3	4	6
EO - External Output	4	5	7
EQ - External Query	3	4	6
ILF - Internal Logical File	7	10	15
EIF - External Interface File	5	7	10

```
UFP = sum(quantidade_simples * peso_simples + ... para cada tipo)
VAF = 0.65 + 0.01 * TDI # TDI = soma dos 14 fatores de influencia (0-70)
AFP = UFP * VAF # ou AFP = UFP se VAF não aplicado
# Botão 'Usar este valor' transfere AFP diretamente para o campo Function Points
```

5.6 Integração com Trello (boards públicos)

A integração foi implementada em `scripts_app/get_public_trello_board.py` e permite importar projetos diretamente de boards Trello sem autenticação:

```
def get_trello_cards_public(url: str) -> pd.DataFrame:
    board_id = re.match(r'https://trello.com/b/([a-zA-Z0-9]+)', url).group(1)
    data = requests.get(f'https://trello.com/b/{board_id}.json').json()

    # Mapeia field_id -> nome canônico (case-insensitive, com aliases)
    field_map = {cf['id']: _resolve_field_name(cf['name'])
                 for cf in data['customFields']}

    # Extrai .value.number de cada customFieldItem por cartão
    rows = [{'project_id': card['name'],
             **{canonical: float(item['value']['number'])
                for item in card['customFieldItems']
                if (canonical := field_map.get(item['idCustomField']))}]}
            for card in data['cards'] if not card['closed']]

    return pd.DataFrame(rows) # project_id + 5 colunas de features
```

Aliases aceitos para nomes de campos (tolerância a erros ortográficos):

Campo Canônico	Aliases Aceitos
function_points	function_points, function points
performance_requirements	performance_requirements, performance requirements

complex_processing	complex_processing, complex processing
installation_ease	installation_ease, instalation_ease, instalation ease, installation ease
additional_complexity_factor	additional_complexity_factor, additional_complexity_factor, additional complexity factor

5.7 Seção de Resultados

Aba de Resultado	Conteúdo
Dados	Tabela com project_id e os 5 campos de entrada; contagem de projetos
Estimativas	Botão 'Calcular'; tabela com horas, dias (h/8), semanas (dias/5); barra visual; download CSV
Visualizações	4 gráficos com títulos e textos descritivos: (1) Distribuição do Esforço Estimado entre Projetos -- histograma por faixa de horas; (2) Tamanho do Projeto x Esforço Estimado -- scatter AFP vs. horas; (3) Complexidade Técnica x Esforço Estimado -- scatter complex_processing vs. horas; (4) Comparação do Esforço entre Projetos -- barras em ordem decrescente de esforço (exibido apenas quando há mais de 1 projeto)

5.8 Bugs Corrigidos Durante o Marco 5

Bug	Causa Raiz	Correção
Predições idênticas (~3981h) para todos os projetos	Notebook de treino re-fitava o Lasso com $\alpha=1.0$ (padrão sklearn), zerando coef	Artefato substituído pelo modelo MLflow correto ($\alpha=0.1$); notebook corrigido para usar <code>grid.best_estimator_</code>
function_points sem efeito na predição	CSV de treino tem fp padronizado, mas app enviava fp raw para API (coef interpretado na escala errada)	Adicionado $(fp-514.86)/516.24$ em <code>preprocess_for_api()</code> usando <code>scaler_maxx.pkl[0]</code>
'Adicionar Projeto' não adicionava projetos	Ausência de <code>st.rerun()</code> após <code>append</code> no <code>session_state</code> ; página não refrescava	Adicionado <code>st.toast() + st.rerun()</code> no callback do botão
Importação Trello sobrescrevia dados manuais a cada rerun	O fetch rodava em todo rerun do Streamlit (fora de callback de botão)	Fetch movido para callback do botão 'Buscar projetos do Trello'; preview antes de confirmar
Upload CSV sobrescrevia dados a cada rerun	Mesmo padrão: escrita fora de callback	Adicionado botão 'Carregar este CSV' com preview de 10 linhas antes de confirmar

Botoes 'Limpar dados' nao funcionavam	Faltava st.rerun() apos limpeza do session_state	Adicionado st.rerun() nos callbacks de limpeza
--	---	---

Processo de Re-treino Manual

O re-treino é um processo manual executado pelo técnico responsável. Não requer interrupção da API em produção durante a execução dos notebooks - apenas no momento do deploy do novo artefato.

Passo	Ação	Arquivo / Ferramenta
1	Atualizar dados brutos com novos projetos (se houver)	model/data/raw/maxx.csv
2	Re-executar EDA: correlações, outliers, PCA	model/notebooks/inferencia/agile_v2/eda.ipynb
3	Salvar artefatos de pré-processamento atualizados	api/artifacts/preprocessing/scaler_pca_features.pkl, pca_2.pkl
4	Re-executar notebook de treino (GridSearchCV + MLflow)	model/notebooks/treino/agile_v2/modelagem-ml.i
5	Verificar coeficientes: não-zero, $\alpha=0.1$	joblib.load('api/artifacts/model/agile_estimator_v2
6	Copiar artefato para api/artifacts/model/ e model/artifacts/model/	Automático via notebook (shutil.copy2)
7	git commit + git push para branch main	Git / GitHub
8	Render refaz o deploy automaticamente (~2 min)	Render.com (webhook GitHub)
9	Verificar health check em produção	GET /health

Script de Verificação Pós-Retreino

```
import joblib, numpy as np

model = joblib.load('api/artifacts/model/agile_estimator_v2.pkl')
lasso = model.steps[-1][1]

print('Alpha:', lasso.alpha)          # deve ser < 1.0
print('Coeficientes:', lasso.coef_)   # NENHUM deve ser exatamente 0.0
print('Intercept:', lasso.intercept_) # esperado ~8.0-9.0

# Teste de sanidade: projetos diferentes -> previsões diferentes
import pandas as pd
p1 = model.predict(pd.DataFrame([{'function_points': -1.0, 'PC1': 0.0, 'PC2': 0.0}]))
p2 = model.predict(pd.DataFrame([{'function_points': 1.0, 'PC1': 1.0, 'PC2': 0.0}]))
assert p1[0] != p2[0], 'ERRO: modelo retorna mesma previsão para inputs diferentes'
print('Previsão projeto pequeno:', np.exp(p1[0]), 'h')
print('Previsão projeto grande: ', np.exp(p2[0]), 'h')
```

Instruções de Operação

Acessar o Sistema em Produção

Recurso	URL / Como acessar
Interface Streamlit	Disponível via Streamlit Cloud ou instância própria
API REST	https://agile-estimator-ofc.onrender.com/predict
Health Check da API	https://agile-estimator-ofc.onrender.com/health
Swagger UI (documentação interativa da API)	https://agile-estimator-ofc.onrender.com/docs
MLflow UI -- histórico de experimentos ML (entrega adicional)	https://agile-estimator-mlflow.onrender.com
Board Trello demo	https://trello.com/b/DKf6KNh2/testeagileestimator

O MLflow UI (<https://agile-estimator-mlflow.onrender.com>) é uma entrega adicional: permite ao cliente visualizar, de forma pública e sem autenticação, todos os experimentos de Machine Learning realizados durante o Marco 2 -- incluindo os 5 algoritmos avaliados, seus hiperparâmetros, métricas e o comparativo que justificou a escolha do Lasso como modelo final.

Usar a Interface Streamlit

1. Acesse o Tutorial (aba Ícone de livro) para entender os campos e a escala Likert.
2. Escolha o modo de entrada: Input Manual, Upload CSV ou Capturar do Trello.
3. Para Input Manual: preencha os 5 campos e clique em 'Adicionar Projeto'. Repita para cada projeto.
4. Para CSV: faça upload do arquivo, visualize o preview e confirme clicando em 'Carregar este CSV'.
5. Para Trello: cole a URL do board público, clique 'Buscar projetos do Trello', confirme com 'Carregar estes projetos'.
6. Na seção de resultados, clique em 'Calcular Esforço Total Estimado'.
7. Visualize os resultados (Estimativas) e os gráficos (Visualizações).
8. Clique em 'Baixar resultados (CSV)' para exportar.

Formato do CSV de Entrada

```
# Colunas obrigatórias (project_id é opcional):
project_id,function_points,performance_requirements,complex_processing,installation_ease,additional_complexity_factor
projeto_alpha,450,3,4,3,3
projeto_beta,800,4,5,2,4
projeto_gamma,220,2,2,5,1
```

Campos Customizados no Trello

Cada cartão do board Trello deve ter campos customizados (Custom Fields) com os nomes abaixo (o sistema aceita variações ortográficas):

- Function Points (ou 'function_points')
- Performance Requirements (ou 'performance_requirements')
- Complex Processing (ou 'complex_processing')
- Installation Ease (ou 'installation_ease', 'instalation_ease')
- Additional Complexity Factor (ou 'additional_complexity_factor')

Todos os campos devem ser do tipo Número no Trello.

Observações Importantes

- Cold start da API: a primeira requisição após 15 min de inatividade leva ~30s. Chamar GET /health antes de calcular estimativas aquece a instância.
- O modelo estima o esforço do PROJETO INTEIRO (não por sprint ou por tarefa). Valores típicos no dataset Maxwell: 583h (min) a 18.500h (max), mediana 4.557h.
- Installation Ease: escala invertida. Valor 5 = muito fácil de instalar (menor esforço nessa dimensão); valor 1 = muito difícil.
- MAPE de ~49%: use as estimativas como baseline para comparação relativa entre projetos, não como compromisso contratual absoluto.

Testes e Validacoes Finais

Esta secao documenta os testes executados para validar os endpoints da API, o pipeline de pre-processamento e os fluxos de integracao do sistema. Os testes automatizados estao implementados em tests/test_api.py e tests/test_pipeline.py e podem ser executados com pytest.

Matriz de Testes

#	Teste	Como Executar	Resultado Esperado
1	GET /health	curl https://agile-estimator-ofc.onrender.com/health	HTTP 200 status: ok model_loaded, inference_ok: true
2	POST /predict (payload valido)	curl -X POST ../predict -d '{"function_points":-0.029,"PC1":0.52,"PC2":-0.	HTTP 200 {"prediction": float > 0} valor entre 100h e 50.000h
3	POST /predict (payload invalido)	curl -X POST ../predict -d '{"function_points": "alto","PC1":0.5,"PC2":0}'	HTTP 422 Pydantic ValidationError -- campo nao e float
4	POST /predict (campos ausentes)	curl -X POST ../predict -d '{"function_points":-0.029}'	HTTP 422 Pydantic ValidationError -- PC1 e PC2 obrigatorios
5	Swagger UI / docs	Abrir https://agile-estimator-ofc.onrender.com/docs no browser	Interface Swagger com /predict e /health documentados e funcionais
6	Streamlit -> API (input manual)	Preencher formulario manual e clicar Calcular Esforco	Estimativa em horas exibida; graficos renderizados; download CSV disponivel
7	Upload CSV	Upload de CSV com colunas obrigatorias; clicar Carregar este CSV	Preview 10 linhas exibido; apos confirmacao, estimativas calculadas em lote
8	Integracao Trello	Colar URL do board publico (trello.com/b/DKf6KNh2/...); Buscar projetos	Cards importados como DataFrame; estimativas calculadas por projeto
9	Docker local	cd api && docker-compose up --build; curl localhost:8000/health	Container sobe em < 60s; /health retorna status ok
10	Sensibilidade do modelo	pytest tests/test_api.py::test_predict_larger_fp_gives_more_effort -v	Projeto com FP maior retorna mais horas -- coeficiente positivo confirmado

Testes Automatizados (pytest)

Os testes automatizados cobrem 20 cenarios divididos em dois modulos:

Modulo	Testes	O que cobre
tests/test_api.py	11 testes	/health (5), /predict valido (5), /predict invalido (3)
tests/test_pipeline.py	9 testes	Artefatos (3), scaler FP (2), shapes (2), modelo (2), ponta a ponta (2)

```
# Instalar dependencias de desenvolvimento
pip install -r requirements-dev.txt
pip install -r api/requirements.txt
```

```
# Rodar todos os testes
```

```
pytest tests/ -v
```

```
# Saida esperada: 20 passed in < 5s
```

Guia de Validação Operacional

Guia rápido para validar os principais fluxos do sistema em ambiente local ou em produção. Siga os passos em ordem para uma validação completa.

1. Subir o Sistema Localmente

Opção A -- Com Docker (recomendado)

```
git clone https://github.com/rodolfobdossantos/agile_estimator_ofc.git
cd agile_estimator_ofc/api
docker-compose up --build
# API disponível em http://localhost:8000
```

Opção B -- Sem Docker

```
cd agile_estimator_ofc/api
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
# API disponível em http://localhost:8000
```

2. Validar a API

```
# Health check
curl http://localhost:8000/health
# Esperado: {"status": "ok", "checks": {"model_loaded": true, ...}}

# Predição com payload válido (FP=500, fatores medianos)
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"function_points": -0.029, "PC1": 0.52, "PC2": -0.31}'
# Esperado: {"prediction": <float entre 1000 e 10000>}

# Swagger UI: abrir http://localhost:8000/docs no browser
```

3. Testar o Streamlit

Inicie a aplicação Streamlit apontando para a API local ou de produção:

```
cd agile_estimator_ofc/streamlit_app
pip install streamlit altair requests joblib scikit-learn
streamlit run app.py
# Acesse http://localhost:8501
```

- Navegue até a aba Input Manual (ícone de formulário)
- Preencha: Function Points=500, demais fatores=3. Clique Adicionar Projeto
- Vá para a aba Estimativas e clique Calcular Esforço Total Estimado
- Resultado esperado: estimativa em horas + dias + semanas + gráficos Altair

4. Testar Upload CSV

Crie um arquivo sample.csv com o conteúdo abaixo e faça upload na aba CSV:

```
project_id,function_points,performance_requirements,complex_processing,installation_ease,additional_complexity
projeto_alpha,450,3,4,3,3
projeto_beta,800,4,5,2,4
```

- Clique em Browse files e selecione o CSV
- Visualize o preview de 10 linhas e clique Carregar este CSV
- Va para Estimativas e calcule -- esperado: 3 projetos com estimativas distintas

5. Validar Integracao Trello

```
# Board de demonstracao publico:  
https://trello.com/b/DKf6KNh2/testeagileestimator
```

- Na aba Trello, cole a URL acima e clique Buscar projetos do Trello
- Confirme com Carregar estes projetos
- Esperado: cards importados com os 5 campos customizados preenchidos
- Va para Estimativas e calcule -- cada card do Trello vira uma linha de resultado

6. Inferencia Ponta a Ponta (Producao)

Para validar o fluxo completo em producao, simule o mesmo calculo que o Streamlit executa internamente antes de chamar a API:

```
# Passo 1: FP raw 500 -> padronizado  
fp_std = (500 - 514.8596) / 516.2373 # = -0.0287  
  
# Passo 2: fatores Likert 3,4,3,3 -> scaler -> PCA  
# (usar scaler_pca_features.pkl + pca_2.pkl)  
# PC1 ~= 0.52, PC2 ~= -0.31 (varia conforme fatores)  
  
# Passo 3: chamar API de producao  
curl -X POST https://agile-estimator-ofc.onrender.com/predict \  
  -H "Content-Type: application/json" \  
  -d '{"function_points": -0.0287, "PC1": 0.52, "PC2": -0.31}'  
  
# Resultado esperado: {"prediction": ~4500.0} # horas-pessoa  
# Range Maxwell: 583h (min) a 18.500h (max) | mediana 4.557h
```

Observacao: a primeira requisicao apos 15 minutos de inatividade leva ~30s (cold start do plano gratuito do Render).
Chame GET /health antes para aquecer a instancia.

Inventario Completo de Artefatos

Artefato	Caminho no Repositorio	Descricao
agile_estimator_v2.pkl	api/artifacts/model/	Lasso Pipeline - alpha=0.1, coef=[0.341,0.167,-0.272]
agile_estimator_v2.pkl	model/artifacts/model/	Copia para repositorio de modelagem
pca_2.pkl	api/artifacts/preprocessing/	PCA n=2 - 70.6% variancia explicada
scaler_pca_features.pkl	api/artifacts/preprocessing/	StandardScaler fitado nos 4 fatores Likert (57 projetos)
scaler_maxx.pkl	api/artifacts/preprocessing/	Scaler Maxwell 9 features; indice 0=function_points
features.json	api/artifacts/metadata/	Metadados: features, pipeline, metricas, dataset
maxx_processed.csv	model/data/processed/	Dataset limpo e transformado (57x14)
modelagem-ml.ipynb	model/notebooks/treino/agile_v2/	Notebook treino com GridSearchCV e MLflow

Agile Estimator v2 | Documentacao Tecnica Detalhada

eda.ipynb	model/notebooks/inferencia/agile_v2/	EDA: correlacoes, outliers, PCA, selecao de features
data_modelling.ipynb	model/notebooks/inferencia/agile_v2/	Analise exploratoria de datasets benchmark
app.py	streamlit_app/	Interface Streamlit - 4 modos entrada, 3 abas resultado
get_public_trello_board.py	streamlit_app/scripts_app/	Importador Trello com alias de campos
main.py	api/app/	FastAPI: rotas /predict, /health
predict.py	api/app/	Logica de inferencia
model_loader.py	api/app/	Carregamento dinamico do .pkl mais recente
schemas.py	api/app/	Pydantic: PredictionInput
Dockerfile	api/	python:3.11-slim; porta via \$PORT
docker-compose.yml	api/	restart: always para reinicio automatico
TECHNICAL_DOCUMENTATIO	docs/	Este documento em formato Markdown
BUSINESS_DOCUMENTATION.	docs/	Documentacao de negocio e proposta comercial
Dockerfile	mlflow_server/	MLflow UI -- deploy no Render como Web Service
README_DEPLOY.md	mlflow_server/	Instrucoes de deploy do MLflow UI no Render
test_api.py	tests/	11 testes automatizados dos endpoints FastAPI
test_pipeline.py	tests/	9 testes do pipeline de pre-processamento
requirements-dev.txt	/	Dependencias de desenvolvimento (pytest, httpx)